

Course Outline: Spec Driven Design

Title: Spec Driven Design **Learnpack:** + Spec Driven Design **Prerequisite:** Estrategias en la comunicación con la IA (Context & Anatomy) **Target Audience:** Beginner developers learning to collaborate effectively with AI code agents **Total Lessons:** 13 **Estimated Total Length:** ~6,500 words **Format:** Conceptual (0% hands-on)

Course Description

Most beginners talk to AI code agents the way they'd talk to a friend: "build me a login page" or "make it look nice." The result? Unpredictable output, constant back-and-forth, and frustration. This course teaches students to replace vague requests with specifications — structured, verifiable documents that tell an AI agent exactly what to build, under what constraints, and how to know when it's done.

Students will learn what a specification is, how Spec Driven Design works as a methodology for delegating to code agents, and how to write specs that consistently produce the results they want. By the end of this course, students will think declaratively — focusing on *what* they want rather than *how* to get it — and will be able to write specs with clear acceptance criteria that serve as contracts between them and their AI agent.

This course builds on the communication strategies covered in the previous learnpack (Context Engineering, word efficiency) and sets the foundation for the next learnpack, where students will learn to codify recurring patterns into reusable rules for their AI coding environment.

Course Philosophy

This course emphasizes:

- **Declarative thinking over imperative thinking:** Focus on describing what you want, not micromanaging how the AI gets there.
 - **Path of least resistance:** Start with the simplest spec that works, then refine only where needed.
 - **Quality of constraint = quality of output:** The problem is almost never the model — it's the quality of the instructions you give it.
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - What Is a Spec?	2	~1,000
02 - Anatomy of a Spec	3	~1,500
03 - Thinking in Specs	3	~1,500
04 - Assessment	2	~1,200
05 - Conclusion	1	~450

Section	Lessons	Estimated Words
Total	13	~6,100

Section 00: Welcome

00.0 Welcome to Spec Driven Design 📖 (~450 words)

Learning Objectives:

- Identify the core problem this course solves: vague AI requests produce unpredictable results
- Recognize the shift from "asking the AI to do things" to "specifying what the AI must deliver"

Content Outline:

- The common frustration: you ask the AI to build something, and it builds something *different*
- Why this happens — the AI isn't broken, the instructions are incomplete
- What this course covers: turning ideas into specifications that code agents can execute reliably
- What students will be able to do by the end: write a spec, define acceptance criteria, and delegate to an agent with confidence
- Brief overview of course sections

Transitions: Opens directly into Section 01, where students will examine the difference between a vague request and a specification.

Key Principles:

- The quality of the AI's output depends on the quality of your input
- A specification is not a longer prompt — it's a different way of thinking

Examples:

- A student asks "build me a to-do app" and gets a random implementation with features they didn't want
 - The same student writes a 6-line spec with constraints and acceptance criteria, and gets exactly what they described
-

Section 01: What Is a Spec?

01.0 What Is a Specification? 📖 (~500 words)

Learning Objectives:

- Define what a specification is in the context of working with AI code agents
- Distinguish between a vague request, a detailed prompt, and a specification
- Explain why vague requests produce unpredictable results (high entropy)

Content Outline:

- The motivation: from "build me this" to "build this under these rules"

- What a specification actually is: a structured description of *what* to build, *under what constraints*, and *how to verify it's correct*
- The spectrum: vague request → detailed prompt → specification
- Why specifications reduce back-and-forth and rework
- A specification is not about writing more — it's about writing the *right* things

Transitions: From the previous welcome lesson where we identified the core problem. Next lesson connects specs to the broader methodology of Spec Driven Design and explains why this matters specifically for AI code agents.

Key Principles:

- A spec answers three questions: What to build? Under what constraints? How do we know it's done?
- Specifications reduce entropy — they narrow the AI's solution space to what you actually want

Examples:

- Vague: "Make me a contact form." Detailed prompt: "Make a contact form with name, email, and message fields using React." Spec: "Build a contact form component. Fields: name (required, text), email (required, valid format), message (required, min 10 chars). On submit, POST to /api/contact. Show success message on 200, error message on failure. No external libraries."
 - Analogy: ordering "a coffee" vs ordering "a double espresso, no sugar, in a ceramic cup" — same effort to say, vastly different results
-

01.1 Spec Driven Design and Code Agents 📖 (~500 words)

Learning Objectives:

- Define Spec Driven Design as a methodology for working with AI code agents
- Explain why code agents specifically benefit from specs (vs. general AI chat)
- Describe how specs change the delegation dynamic: from "hoping it works" to "verifying against criteria"

Content Outline:

- What is Spec Driven Design? A workflow where you write the specification *before* asking the agent to code anything
- Why code agents need specs more than chat-based AI: agents execute autonomously, so unclear instructions compound into larger errors
- The delegation shift: specs let you delegate confidently because you defined what "done" looks like before work began
- The relationship: spec = contract, agent = contractor, acceptance criteria = inspection checklist
- A good spec lets you review less and rework less

Transitions: Students now understand what a spec is and why it matters for code agents. Section 02 dives into the concrete structure — what actually goes inside a well-made spec.

Key Principles:

- Define the spec before the agent writes a single line of code

- The problem is rarely the model — it's the quality of the framing of the work
- A good spec allows you to delegate better, review less, and reduce rework

Examples:

- Without SDD: student asks agent to "build a dashboard," gets a generic layout, spends 45 minutes going back and forth fixing it. With SDD: student writes a spec defining sections, data sources, and layout constraints — agent delivers on the first pass.
 - The contractor analogy: you wouldn't tell a builder "make me a house" without blueprints. A spec is your blueprint for the code agent.
-

Section 02: Anatomy of a Spec

02.0 The Building Blocks of a Spec 📖 (~500 words)

Learning Objectives:

- Identify the core components of a well-structured specification
- Distinguish between necessary context and noise in a spec
- Construct the skeleton of a spec from a project idea

Content Outline:

- The anatomy: goal, scope, constraints, acceptance criteria, and context
- Goal: one sentence — what is the agent building?
- Scope: what's included and (critically) what's excluded
- Constraints: technical boundaries (stack, libraries, file structure, patterns)
- Acceptance criteria: how to verify the result (covered in depth next lesson)
- Context: only the information the agent *needs* to do the work — nothing more
- Why every part matters: removing any one creates ambiguity

Transitions: From Section 01 where students learned *why* specs matter. This lesson gives them the structural framework. Next lesson zooms into the most critical component: acceptance criteria.

Key Principles:

- Every component of a spec exists to reduce ambiguity in one specific dimension
- Prompts don't need to be exaggeratedly long — they need the information that's truly necessary for the model to understand what you want

Examples:

- A spec for a "user profile card" component showing all five parts filled in: goal (render user info), scope (display only, no edit), constraints (React, Tailwind, no external libs), acceptance criteria (shows name, avatar, bio; handles missing avatar gracefully), context (receives user object as prop)
 - A bad spec that has 3 paragraphs of company history but no constraints — looks long, but the agent still guesses at everything that matters
-

02.1 Acceptance Criteria as a Contract 📖 (~500 words)

Learning Objectives:

- Write acceptance criteria that are specific, testable, and complete
- Explain why acceptance criteria function as a contract between student and agent
- Differentiate between vague success conditions and verifiable criteria

Content Outline:

- What acceptance criteria are: a checklist of conditions that must be true for the work to be considered done
- Why "contract" is the right metaphor: the agent builds to satisfy these criteria, you review against them
- The formula: "Given [context], when [action], then [expected result]"
- How specific is specific enough? If you can't objectively check it, it's not a criterion
- Common mistake: writing criteria that describe feelings ("it should look nice") instead of facts ("it should use 16px spacing between cards")

Transitions: Students can now write the most critical part of a spec. Next lesson covers the remaining best practices that make specs agent-ready.

Key Principles:

- If you can't verify it objectively, it's not an acceptance criterion — it's a wish
- Acceptance criteria are how you check the agent's work without reading every line of code

Examples:

- Bad: "The form should work properly." Good: "Given a user submits the form with all required fields filled, then a POST request is sent to /api/contact and a success message is displayed."
- Bad: "The page should be responsive." Good: "On viewports below 768px, the grid switches from 3 columns to 1 column."

02.2 Best Practices for Agent-Ready Specs 📖 (~500 words)

Learning Objectives:

- Apply the principle "small, clear, and precise" when writing specifications
- Identify when a spec is too broad and needs to be split
- Write specs that minimize rework by anticipating common agent mistakes

Content Outline:

- Best practice 1: Define specifications *before* you start developing — don't spec retroactively
- Best practice 2: Keep specs small, clear, and precise — one spec per feature or component, not per page or app
- Best practice 3: Mention specific files the agent should read or modify
- Best practice 4: Include what to *not* do (negative constraints prevent common agent defaults you don't want)
- Best practice 5: Use the vocabulary the agent understands — technical terms over subjective descriptions

- How to know your spec is ready: could a different agent (or a different person) read this spec and produce the same result?

Transitions: Students now know how to write a complete, well-structured spec. Section 03 shifts from *what goes in a spec* to *how to think when writing specs* — the mental models that make spec-writing intuitive.

Key Principles:

- Small specs beat big specs — one focused spec per component or feature
- A good spec is one that a stranger could read and produce the same result you'd expect

Examples:

- A spec that tries to cover an entire landing page (Navbar + Hero + Features + Footer) in one document — too broad, should be 4 separate specs
 - A spec that says "use the existing design system" vs one that says "use Tailwind, follow the spacing scale in tailwind.config.ts, reference the Button component in /components/ui/Button.tsx" — the second one the agent can actually act on
-

Section 03: Thinking in Specs

03.0 Tell the AI What, Not How 📖 (~500 words)

Learning Objectives:

- Distinguish between imperative thinking ("do step 1, then step 2") and declarative thinking ("here's what the result should look like")
- Apply declarative thinking when writing specifications for code agents
- Recognize when a spec is slipping into imperative micromanagement

Content Outline:

- The shift: from "how to do it" (imperative) to "what I want to get" (declarative)
- Why imperative specs fail with AI agents: you're guessing at implementation steps the agent is better at choosing
- Declarative specs describe the *destination*, not the *route* — the agent figures out the route
- When imperative is okay: explicit constraints ("use this library," "don't use class components") are fine because they narrow options, they don't dictate steps
- The test: if your spec reads like a tutorial, you're being imperative. If it reads like a contract, you're being declarative.

Transitions: From Section 02 where students learned spec structure. This lesson introduces the *mindset*. Next lesson teaches a practical pattern for applying this thinking to complex projects.

Key Principles:

- Declarative thinking: describe the *what*, let the agent decide the *how*
- Constraints narrow options (good). Step-by-step instructions micromanage execution (bad).

Examples:

- Imperative spec: "First create a React component. Then add a useState hook for the form data. Then add an onChange handler. Then add a handleSubmit function that calls fetch..." Declarative spec: "Build a contact form component in React. It should manage its own state and POST to /api/contact on submit."
 - Analogy: telling a taxi driver turn-by-turn directions vs giving them the address. The address is faster and less error-prone.
-

03.1 The Matrioshka Pattern 📖 (~500 words)

Learning Objectives:

- Apply the Matrioshka pattern to decompose a large project into a sequence of small, independent specs
- Explain why writing one spec per component produces better results than one giant spec for the entire page
- Order specs from outermost container to innermost detail (or by logical section)

Content Outline:

- The pattern: never spec the entire page at once — spec component by component, from outside in or section by section
- Why it works: each spec stays small and focused, each result can be verified independently, each mistake is contained
- The naming: like a Matrioshka (Russian nesting doll), you open the outer layer first and work inward
- How to apply it: identify the top-level containers, write a spec for each, then go one level deeper
- This is the Matrioshka applied to *spec writing* — not to prompting in general. The goal is better specs, not just smaller prompts.
- When to stop nesting: if the component is simple enough to spec in one short paragraph, don't split further

Transitions: Students now have both the structural knowledge (Section 02) and the thinking tools (declarative thinking + Matrioshka) to write good specs. Next lesson covers what goes wrong — the common mistakes that undo good intentions.

Key Principles:

- One spec per component or feature — never one spec for the whole thing
- Work outside-in: layout first, then sections, then individual components

Examples:

- Bad: one 40-line spec for an entire e-commerce product page (hero, gallery, reviews, related products, cart button). Good: separate specs for ProductHero, ProductGallery, ReviewsList, RelatedProducts, AddToCartButton — each one 5-8 lines.
 - A student specs a dashboard: first the layout (sidebar + main content area), then the sidebar nav, then each widget in the main area — each as its own focused spec.
-

03.2 Common Spec Mistakes 📖 (~500 words)

Learning Objectives:

- Identify the four most common spec anti-patterns and explain why each one produces poor results
- Diagnose which anti-pattern a given bad spec is exhibiting
- Rewrite a flawed spec to eliminate the anti-pattern

Content Outline:

- Anti-pattern 1 — The One-Shot Wonder: trying to generate an entire app (Home, Detail, Checkout) in a single giant spec. Why it fails: too many decisions at once, compounding errors, impossible to verify.
- Anti-pattern 2 — The Amnesia Assumption: assuming the AI remembers everything from earlier in the project (e.g., which libraries you picked 3 prompts ago). Why it fails: agents don't carry context between sessions by default.
- Anti-pattern 3 — The Backstory Dump: writing 4 paragraphs of company history that don't affect the code. Why it fails: wastes tokens and dilutes the agent's attention from what actually matters.
- Anti-pattern 4 — The Guessing Prompt: "Make me a nice page for a store." Why it fails: maximum entropy — the agent has to guess at every decision, so it hallucinates random styles and structures.
- For each: the anti-pattern, why it's tempting, what goes wrong, and the fix.

Transitions: Students now have the complete toolkit: structure, best practices, thinking patterns, and awareness of common pitfalls. Section 04 puts it all together with synthesis and assessment.

Key Principles:

- Every anti-pattern is a form of missing constraint — the fix is always to add the right specificity
- These mistakes are tempting because they feel faster. They always cost more time in rework.

Examples:

- One-Shot Wonder: "Build me an Airbnb clone with search, listings, detail view, booking, and user profiles" → should be 5+ separate specs.
- Amnesia Assumption: "Now add the payment form using the same library" (but the agent doesn't remember which library) → the spec should re-state: "using Stripe.js, as configured in /lib/stripe.ts."
- Backstory Dump: "We are a sustainable fashion company founded in 2019 in Barcelona with a mission to..." → cut to: "E-commerce site. Stack: Next.js + Tailwind. Product catalog with filter by category."
- Guessing Prompt: "Make me a nice landing page for my startup" → "Build a landing page with: Hero (headline + CTA button), Features (3-column grid), and Footer. Use Tailwind. Color scheme: navy (#1e3a5f) and white."

Section 04: Assessment

04.0 Good Specs vs Weak Specs (~500 words)

Learning Objectives:

- Evaluate a specification and identify its strengths and weaknesses
- Apply course concepts (anatomy, acceptance criteria, declarative thinking, anti-patterns) to critique real specs
- Transform a weak spec into a strong one by applying targeted fixes

Content Outline:

- Walkthrough: taking a vague idea ("I want a blog") and building it into a proper spec step by step — goal, scope, constraints, acceptance criteria
- Side-by-side: the same feature specified poorly vs well — what changes and why each change matters
- Spotting weakness: a gallery of 3-4 specs with subtle problems — students identify which anti-pattern is present and what's missing
- The revision mindset: fixing a spec is cheaper than fixing bad code — always review the spec before handing it to the agent

Transitions: From Section 03 where students learned thinking patterns and anti-patterns. This lesson synthesizes everything into practical evaluation skill. Next lesson is the formal assessment.

Key Principles:

- Reading specs critically is as important as writing them
- A weak spec is always cheaper to fix than the bad code it produces

Examples:

- Walkthrough: "I want a blog" → adds goal ("display posts with title, date, and preview") → adds scope ("read-only, no admin panel") → adds constraints ("Next.js, markdown files in /content") → adds acceptance criteria ("homepage shows 10 most recent posts, each links to full post page, posts render markdown to HTML")
 - Side-by-side: a "user settings page" spec with no acceptance criteria vs the same spec with 5 clear criteria — the first generates a random form, the second generates exactly what was needed
-

04.1 Spec Driven Design Assessment 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of spec structure, acceptance criteria, declarative thinking, and anti-pattern recognition
- Apply course concepts to evaluate and improve specifications in realistic scenarios

Question Format: Scenario-based (5 questions)

Questions:

Q1 — Identifying a Specification A student sends this to their AI code agent: "Build a React component that shows a list of users. Each user should display their name, email, and a delete button. Use Tailwind for styling. The delete button should call DELETE /api/users/:id and remove the user from the list without a page refresh." Which of the following best describes this text?

- A) A vague request
- B) A detailed prompt
- C) A specification
- D) An acceptance criterion

Correct: C — It has a clear goal, constraints (React, Tailwind), and verifiable behavior (DELETE endpoint, no refresh). It's missing explicit acceptance criteria as a separate section, but the behavioral descriptions function as testable criteria.

Q2 — Acceptance Criteria A student writes this acceptance criterion for a search bar component: "The search should work well and be fast." What is the main problem with this criterion?

- A) It's too short
- B) It's not objectively verifiable
- C) It doesn't mention the technology stack
- D) It should be written in code, not English

Correct: B — "Work well" and "fast" are subjective. A verifiable version would be: "Given a user types in the search field, results matching the query appear within 500ms without a page reload."

Q3 — Declarative vs Imperative Which of the following is a declarative spec instruction?

- A) "First create a useState hook, then add an onChange handler, then validate the input"
- B) "The form should validate email format on blur and show an inline error message below the field"
- C) "Step 1: import React. Step 2: create a functional component. Step 3: add a return statement"
- D) "Use a for loop to iterate through the array and push each item to a new array"

Correct: B — It describes the desired behavior (what) without dictating implementation steps (how). The agent decides how to implement validation and error display.

Q4 — Anti-Pattern Recognition A student writes the following spec: "Build a complete e-commerce site. Include a homepage with featured products, a product detail page with reviews, a shopping cart with quantity controls, a checkout flow with Stripe, user authentication with email and Google, an admin dashboard for managing products, and a review moderation system." Which anti-pattern is this?

- A) The Backstory Dump
- B) The Amnesia Assumption
- C) The One-Shot Wonder
- D) The Guessing Prompt

Correct: C — This tries to build an entire application in a single spec. It should be decomposed into many small specs using the Matrioshka pattern.

Q5 — Fixing a Weak Spec A student shows you this spec: "Our company was founded in 2020 by two developers passionate about education. We believe everyone deserves access to quality learning. Our platform has over 10,000 users across Latin America. We need a new dashboard page that looks modern and professional." What is the most impactful improvement to this spec?

- A) Add more company background to give the agent better context
- B) Remove the company background and add specific constraints, layout structure, and acceptance criteria
- C) Make it longer by describing every visual detail
- D) Translate it to code comments instead of natural language

Correct: B — The company history is a Backstory Dump that wastes tokens and adds zero useful information for the agent. Replacing it with constraints (stack, layout) and acceptance criteria (what sections, what data) would

make this spec actionable.

Evaluation Criteria:

- 5/5: Full mastery — ready to write specs independently
 - 4/5: Strong understanding — minor review recommended
 - 3/5: Adequate — revisit acceptance criteria and anti-patterns
 - 2/5 or below: Re-study Sections 01-03 before proceeding
-

Section 05: Conclusion

05.0 Next Steps: From Specs to Rules (~450 words)

Content Outline:

- Course recap: what students accomplished — they can now write structured specifications with clear acceptance criteria, think declaratively, decompose projects using the Matrioshka pattern, and spot common spec mistakes
- Connection to bigger picture: specs are one-time instructions for a single task — but what about patterns you want the agent to follow *every* time? That's where rules come in, which is exactly what the next learnpack covers
- Next steps: in the next learnpack (¿Qué son las reglas para una IA de programación?), students will learn to turn recurring spec patterns into persistent rules that configure their AI coding environment
- Continued learning: practice writing specs for every feature before asking the agent to build it — even small ones. The habit matters more than perfection.
- Encouragement: the shift from "asking" to "specifying" is the single biggest upgrade in how students work with AI. Every spec they write from now on saves time, reduces frustration, and makes them a better collaborator with their code agent.

Note: This is conclusion only — no theory, exercises, or assessment.
